LeetCode Practice

14 Medium: Math

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

14
MEDIUM

Patterns (in order)

1. Math (14)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

14 questions | tap a title to jump to it

Medium questions (14)	Go >
Math (14)	**Go >**
M 1. Reverse Integer	LC 7
M 2. Multiply Strings	LC 43
M 3. Pow(x, n)	LC 50
M 4. Factorial Trailing Zeroes	LC 172
M 5. Count Primes	LC 204
M 6. Insert Delete GetRandom O(1)	LC 380
M 7. Continuous Subarray Sum	LC 523
M 8. Valid Square	LC 593
M 9. Minimum Area Rectangle II	LC 963
M 10. Convert to Base -2	LC 1017
M 11. Find the Winner of the Circular Game	LC 1823
M 12. Count Good Numbers	LC 1922
M 13. Number of Zero-Filled Subarrays	LC 2348
M 14. Find the Number of Distinct Colors Among the Balls	LC 3160

Medium questions (14)

MEDIUM | TOPIC 1 OF 1

Math

14 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Reverse Integer LC 7
2. Multiply Strings LC 43
3. Pow(x, n) LC 50
4. Factorial Trailing Zeroes LC 172
5. Count Primes LC 204
6. Insert Delete GetRandom O(1) LC 380
7. Continuous Subarray Sum LC 523
8. Valid Square LC 593
9. Minimum Area Rectangle II LC 963
10. Convert to Base -2 LC 1017
11. Find the Winner of the Circular Game LC 1823
12. Count Good Numbers LC 1922
13. Number of Zero-Filled Subarrays LC 2348
14. Find the Number of Distinct Colors Among the Balls LC 3160

MEDIUM | Math | LeetCode #7 | 1 of 14 | 1/14

Reverse Integer

QUESTION

Given a signed 32-bit integer x, return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range [-2^{31}, $2^{31} - 1$], then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1	
Input	`x = 123`
Output	`321`

Example 2	
Input	`x = -123`
Output	`-321`

Example 3	
Input	`x = 120`
Output	`21`

CONSTRAINTS

- $2^{31} <= x <= 2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Reverse the digits of a signed 32-bit integer. Return 0 if overflow.

Assume environment doesn't allow 64-bit integers. Right?"

"A few quick questions:

x=120 -> 21 (trailing zero becomes leading zero -> dropped) ok

x=-123 -> -321 ok. Overflow range: [-2^31, 2^31-1]."

"Let me walk: x=123 -> 321 ok. x=-123 -> -321 ok. x=1534236469 -> 0 (overflow) ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
turn x into a string, reverse it (keep the sign),
    turn it back into a number
if it is outside the 32-bit range:
    return 0
```

TIME	SPACE
O(d)	O(d)

PHASE 2 BRUTE FORCE

NOTE

Not slow in time (d digits). The cost is the string copy, O(d) space. Popping digits with % 10 and building the result with * 10 uses O(1) space but must check overflow before every step.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Build reversed integer digit by digit, checking 32-bit overflow — O(d). Walk-through:"

result=0. While x!=0:

`digit = x % 10 (in Python, handle negatives with abs then re-apply sign).`

x //= 10.

If result*10 + digit would overflow [-2^{31}, $2^{31}-1$]: return 0.

`result = result*10 + digit.`

Return result with original sign.

WHY check before multiplying: result*10 + digit could exceed INT_MAX=2147483647

or INT_MIN=-2147483648. Check: result > (2^{31}-1)//10 OR (result==(2^{31}-1)//10 AND digit>7).

WHY handle Python differently: Python % never gives negative results; pop digit = x%10,

then x //= 10 works directly for positive x (work with abs(x), restore sign).

Time: O(logn). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _7_ReverseInteger:
    def reverse(self, x):
        INT_MAX, INT_MIN = 2**31 - 1, -(2**31)
        sign = -1 if x < 0 else 1
        x = abs(x)
        result = 0
        while x:
            digit = x % 10; x //= 10
            if result > (INT_MAX - digit) // 10:
                return 0
            result = result * 10 + digit
        return sign * result
```

PHASE 5 TEST & DEBUG

"Let me trace reverse(123)."

result=0. x=123.

pop=3: result=0*10+3=3. x=12. pop=2: result=32. x=1. pop=1: result=321. x=0.

321 in [INT_MIN, INT_MAX] -> return 321 ok

`reverse(-123): handle sign. reverse(100): result=1 (trailing zeros drop) ok`

PHASE 5 TEST & DEBUG

INT32 overflow check: before result=result*10+pop, check if result>INT_MAX//10 or (==INT_MAX//10 and pop>7). Return 0 if overflow.

n=-2147483648: reverse -> overflow. Return 0 ok

WHY check overflow before multiply: prevent UB. result>INT_MAX//10 OR (==INT_MAX//10 and pop>7) -> overflow.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(d)	**O(1)**
d = number of digits	integer variables

In Java/C++ the overflow check is critical — Python big-ints hide the issue.

Follow-up: Palindrome Number (#9) — compare reversed half to original.

Follow-up: if 64-bit is allowed, just reverse and check range.

MEDIUM | Math | LeetCode #43 | 2 of 14 | 2/14

Multiply Strings

QUESTION

Given two non-negative integers num1 and num2 represented as strings, return the product of num1 and num2, also represented as a string.

Note: You must not use any built-in BigInteger library or convert the inputs to integer directly.

Example 1	
Input	num1 = "2", num2 = "3"
Output	"6"

Example 2	
Input	num1 = "123", num2 = "456"
Output	"56088"

CONSTRAINTS

- 1 <= num1.length, num2.length <= 200
- num1 and num2 consist of digits only.
- Both num1 and num2 do not contain any leading zero, except the number 0 itself.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Multiply two numbers given as strings, return the product as a string.

Cannot convert to int directly (numbers can be arbitrarily large). Right?"

"A few quick questions:

Either input could be '0'? Yes — handle that. No leading zeros in output except '0' itself."

"Let me walk: num1='123', num2='456' -> '56088' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result = '0'
add num1 to result num2 times (num2 as a number),
    using string addition
```

TIME	SPACE
O(10^m * n)	O(n + m)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Repeated addition runs num2 times, which is exponential in its digit count m. Grade-school multiplication adds one digit-by-digit partial product per digit: O(n * m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Grade-school multiplication on digit arrays — O(mn). Walk-through:"

`result = [0] * (len(num1)+len(num2)).`

For i from len(num1)-1 down to 0:

For j from len(num2)-1 down to 0:

`product = (num1[i]-'0') * (num2[j]-'0').`

`pos1 = i+j. pos2 = i+j+1 (position in result array).`

`sum_ = product + result[pos2].`

`result[pos2] = sum_ % 10.`

`result[pos1] += sum_ // 10 (carry into next position).`

Strip leading zeros. Return ".join(str(d) for d in result).

WHY i+j and i+j+1: multiplying digit at position i (from right in num1) and j (from right

in num2) contributes to position (i+j) in the result (like grade-school alignment).

The ones digit goes to pos2=i+j+1; carry goes to pos1=i+j.

Time: O(mn). Space: O(m+n)."

Time: O(n) — optimal single-pass or O(n²) DP. Linear preferred.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _43_MultiplyStrings:
    def multiply(self, num1, num2):
        if num1 == '0' or num2 == '0':
            return '0'
        m, n = len(num1), len(num2)
        result = [0] * (m + n)
        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                product = int(num1[i]) * int(num2[j])
                pos_low, pos_high = i + j + 1, i + j
                total = product + result[pos_low]
                result[pos_low] = total % 10
                result[pos_high] += total // 10
        result_str = ''.join(map(str, result)).lstrip('0')
        return result_str
```

PHASE 5 TEST & DEBUG

"Let me trace multiply('123','456')."

Simulate grade-school multiplication. result=[0]*5 (len=3+3-1=5... actually 6).

result=[0,0,0,0,0,0].

i=2(3),j=2(6): pos=4. 3*6=18. result[4]+=18->18.

i=2(3),j=1(5): pos=3. 3*5=15. result[3]+=15.

i=2(3),j=0(4): pos=2. 3*4=12. result[2]+=12.

PHASE 5 TEST & DEBUG

i=1(2),j=2(6): pos=3. 2*6=12. result[3]+=12->27.

... After all: result=[0,0,1+0,6+2+1,3+0+2,8]. Then carry propagation.

Carry from pos4: 56%10=6->pos4=6,carry=5. pos3: (15+12+12+5)%10=4,carry=4. ... final='56088' ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	O(m+n)

Follow-up: Add Strings (#415) — simpler single-pass addition, same string-as-number idea.

Follow-up: if numbers can be negative, track sign separately and strip before multiplying.

MEDIUM | Math | LeetCode #50 | 3 of 14 | 3/14

Pow(x, n)

QUESTION

Implement pow(x, n), which calculates x raised to the power n (i.e., x^n).

Example 1	
Input	`x = 2.00000, n = 10`
Output	`1024.00000`

Example 2	
Input	`x = 2.10000, n = 3`
Output	`9.26100`

Example 3	
Input	`x = 2.00000, n = -2`
Output	`0.25000`
Explanation	$2^{-2} = 1/2^2 = 1/4 = 0.25$

CONSTRAINTS

- $100.0 < x < 100.0$
- $2^{31} <= n <= 2^{31}-1$
- n is an integer.
- Either x is not zero or n > 0.
- $10^4 <= x^n <= 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement pow(x, n) — x raised to the power n. n can be negative. Right?"

"A few quick questions:

x can be negative? Yes. n can be -2^31? Yes — handle with care.

x=0.0, n<0 is undefined per constraints (x won't be 0 when n<0)."

"Let me walk: pow(2.0,10)->1024.0 ok. pow(2.0,-2)->0.25 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result = 1
for _ in range(n):
    result = result * x
    if n < 0:
        return 1 / result
```

TIME	SPACE
O(n)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

n multiplications, and n can be 2^31. Squaring (x^n = (x^(n/2))^2) is O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is the O(|n|) multiplications.

Fast exponentiation: if n even: x^n = (x²)^(n/2). If n odd: x^n = x * x^(n-1).

Halves n each step -> O(log n) multiplications.

Time: O(log n). Space: O(1) iterative."

Key insight: fast exponentiation halves n at each step by squaring x — O(n) multiplications collapse to O(log n) because x^n = (x²)^(n/2).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _50_PowXN:
    def myPow(self, x, n):
        if n < 0:
            x = 1 / x
            n = -n
        result = 1
        while n:
            if n % 2 == 1:
                result *= x
            x *= x
            n //= 2
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace myPow(2.0, 10)."

Fast exponentiation. n=10 (even): result = myPow(2.0,5)². myPow(2.0,5).

n=5 (odd): result = 2.0 * myPow(2.0,4)². myPow(2.0,4).

n=4->myPow(2.0,2)²->myPow(2.0,1)²->2.0*myPow(2.0,0)²=2.0*1 =2.0.

Unwind: pow(2,1)=2.0. pow(2,2)=4.0. pow(2,4)=16.0. pow(2,5)=2.0*16=32.0. pow(2,10)=32²=1024.0 ok

n=-2: use n=2, then result=1/result.

`myPow(2,-3)=1/myPow(2,3)=1/8=0.125. Handle negative exponent by inverting.`

WHY halving is O(log n): exponent halved each recursive call -> depth = log(n) levels.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	**O(1)**
halve the exponent each step	iterative

Recursive version is O(log n) time but O(log n) stack space.

Follow-up: matrix exponentiation — Fibonacci in O(log n) using the same squaring.

Follow-up: modular exponentiation — same logic with (result * x) % mod at each step.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Math | LeetCode #172 | 4 of 14 | 4/14

Factorial Trailing Zeroes

QUESTION

Given an integer n, return the number of trailing zeroes in n!.

Note that n! = n * (n - 1) * (n - 2) * ... * 3 * 2 * 1.

Example 1	
Input	`n = 3`
Output	`0`
Explanation	*3! = 6, no trailing zero.*

Example 2	
Input	`n = 5`
Output	`1`
Explanation	*5! = 120, one trailing zero.*

Example 3	
Input	`n = 0`
Output	`0`

CONSTRAINTS

- $0 <= n <= 10^4$
- Follow up: Could you write a solution that works in logarithmic time complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count trailing zeros in n!. Each zero comes from a factor of 10 = 2×5. 5s are the bottleneck. Right?"

"A few quick questions:

Count how many times 5 divides n! ? Yes."

"Let me walk: n=25 -> 25//5=5, 25//25=1 -> 6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
factorial = n! computed with big-number
    multiplication
count the zeros at the end of its decimal form
```

TIME	SPACE
O(n^2 log n)	O(n log n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

n! has about n log n digits, and multiplying them n times is expensive. Each trailing zero is a factor of 10 = 2 * 5, so counting factors of 5 (n/5 + n/25 + ...) is O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count factors of 5 — O(log n). Walk-through:"

Trailing zeros = min(factors of 2, factors of 5) in n!.

Factors of 2 always > factors of 5 -> count only factors of 5.

Multiples of 5 contribute 1 factor of 5. Multiples of 25 contribute 2 (one extra). Etc.

`count = n//5 + n//25 + n//125 + ... until 5^k > n.`

WHY this sum: n//5 counts all multiples of 5. n//25 counts those that contribute an EXTRA

factor (already counted once in n//5). n//125 for triple factors, etc.

Example: n=30 -> 30//5=6 + 30//25=1 = 7. (5,10,15,20,25,30 -> but 25=5² gives extra) ok

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _172_FactorialTrailingZeroes:
    def trailingZeroes(self, n):
        count = 0
        while n >= 5:
            n //= 5
            count += n
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace trailingZeroes(25)."

Count factors of 5 in 25!. Every 5th number contributes one 5. Every 25th contributes two.

25//5=5 (numbers 5,10,15,20,25). 25//25=1 (number 25 contributes extra 5). 25//125=0.

Total=5+1=6 ok (25!=...×10×15×20×25×...->six trailing zeros).

`trailingZeroes(100): 100//5=20, 100//25=4, 100//125=0. Total=24 ok`

WHY divide by powers of 5: 2s always outnumber 5s, so only 5s matter for trailing zeros.

`trailingZeroes(0)=0 ok (0!=1, no zeros).`

WHY not count 2s: factors of 2 are far more abundant (every even number) -> always more 2s than 5s. Only 5s limit trailing zeros.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

Follow-up: count trailing zeros in base b: count factors of smallest prime in b.

Follow-up: count factors of 2 — same algorithm; but factors of 5 are always fewer.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Math | LeetCode #204 | 5 of 14 | 5/14

Count Primes

QUESTION

Given an integer n, return the number of prime numbers that are strictly less than n.

Example 1	
Input	`n = 10`
Output	`4`
Explanation	*There are 4 prime numbers less than 10, they are 2, 3, 5, 7.*

Example 2	
Input	`n = 0`
Output	`0`

Example 3	
Input	`n = 1`
Output	`0`

CONSTRAINTS

- $0 <= n <= 5 * 10^6$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count prime numbers less than n (strictly less). Right?"

"A few quick questions:

n=0 or n=1 -> 0? Yes. n=2 -> 0 (no prime < 2)? Yes."

"Let me walk: n=10 -> primes 2,3,5,7 -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for n in ns:
    if no divisor from 2 to sqrt(i) divides it:
        count = count + 1
```

TIME	SPACE
O(n * sqrt(n))	O(1)

WHY IT IS SLOW

Trial-divides every number separately. The sieve of Eratosthenes crosses out multiples of each prime once: O(n log log n) time, O(n) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sieve of Eratosthenes — O(n log log n). Walk-through:"

`is_prime = [True] * n. Mark 0 and 1 as not prime.`

For p from 2 to sqrt(n):

If is_prime[p]: mark all multiples p^2, p^2+p, p^2+2p, ... as not prime.

(Start from p^2 because smaller multiples of p were already marked by smaller primes.)

Count remaining True values.

WHY start marking from p^2: p*2, p*3, ..., p*(p-1) were already marked by 2, 3, ..., (p-1).

WHY O(n log log n): each prime p marks n/p numbers -> sum over primes ~ n * sum(1/p) ~ n log log n.

Time: O(n log log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _204_CountPrimes:
    def countPrimes(self, n):
        if n < 2:
            return 0
        is_prime = [True] * n
        is_prime[0] = is_prime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if is_prime[i]:
                for j in range(i*i, n, i):
                    is_prime[j] = False
        return sum(is_prime)
```

PHASE 5 TEST & DEBUG

"Let me trace countPrimes(10) using Sieve of Eratosthenes."

`sieve = [True]*10; mark sieve[0]=sieve[1]=False`

p=2: mark 4,6,8 -> False; sieve=[F,F,T,T,F,T,F,T,F,F]

p=3: mark 6(done),9 -> sieve[9]=False; sieve=[F,F,T,T,F,T,F,T,F,F]

p=4: already False, skip; p=5: $5^2=25>=10$, stop outer loop

Count True: indices 2,3,5,7 -> 4 primes ok

WHY start inner loop at p^2: smaller multiples already marked by earlier primes.

Edge: n<=2 -> return 0 (no primes less than 2).

Time O(n log log n); space O(n) for sieve array.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log log n)	O(n)

Follow-up: Count Primes in range [l,r] — segmented sieve for large ranges.

Follow-up: find nth prime — sieve of appropriate size.

MEDIUM | Math | LeetCode #380 | 6 of 14 | 6/14

Insert Delete GetRandom O(1)

QUESTION

Implement the RandomizedSet class:

CONSTRAINTS

- RandomizedSet() Initializes the RandomizedSet object.
- bool insert(int val) Inserts an item val into the set if not present. Returns true if the item was not present, false otherwise.
- bool remove(int val) Removes an item val from the set if present. Returns true if the item was present, false otherwise.
- int getRandom() Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the same probability of being returned.

You must implement the functions of the class such that each function works in average O(1) time complexity.

- 2^{31} <= val <= 2^{31} - 1
- At most 2 * 10^5 calls will be made to insert, remove, and getRandom.
- There will be at least one element in the data structure when getRandom is called.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a set supporting insert(val)->bool, remove(val)->bool, and getRandom()->val

all in average O(1). getRandom returns each element with equal probability. Right?"

"A few quick questions:

insert on existing key -> return false? Yes. remove on missing key -> false? Yes.

getRandom won't be called on empty set? Guaranteed."

"Let me walk: insert(1)->T, insert(2)->T, getRandom()->1 or 2 with equal prob ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def add(x):
    if x is not already in the list (scan):
        append it
def delete(x):
    find it by scanning and remove it (shift)
def getRandom():
    pick a random list index
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n) for add and delete, O(1) for getRandom	O(n)

WHY IT IS SLOW

Add and delete scan and shift the list. A list plus a hash map from value to index (delete by swapping with the last element) makes all three O(1).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) list conversion for getRandom.

Array + HashMap: array holds all values, map holds {val->array_index}.

getRandom: pick a random array index — O(1).

remove: swap target with last element, update map, pop last — O(1).

Time: O(1) all ops. Space: O(n)."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import random
class _380_InsertDeleteGetRandom:
    def __init__(self):
        self._vals = []
        self._idx = {}

    def insert(self, val):
        if val in self._idx: return False
        self._idx[val] = len(self._vals)
        self._vals.append(val)
        return True

    def remove(self, val):
        if val not in self._idx: return False
        last = self._vals[-1]
        i = self._idx[val]
        self._vals[i] = last
        self._idx[last] = i
        self._vals.pop()
        del self._idx[val]
        return True

    def getRandom(self):
        return random.choice(self._vals)
```

PHASE 5 TEST & DEBUG

"Let me trace RandomizedSet insert(1),insert(2),remove(1),getRandom()."

Use list + dict{val->index}

`insert(1): list=[1], dict={1:0}`

`insert(2): list=[1,2], dict={1:0,2:1}`

`remove(1): swap with last -> list=[2,1] wait: swap dict[1]=0 with list[-1]=2`

list=[2], dict={2:0}. Remove last. list=[2], dict={2:0} ok

`getRandom(): random.choice([2]) -> 2 ok`

WHY swap-with-last: O(1) deletion from list (avoid shifting); dict tracks indices.

Edge: remove non-existent -> return False; insert existing -> return False.

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops O(1) average: array ops + dict ops. Space O(n).

Edge case: when removing the last element, swap is a no-op (last==val).

Follow-up: Insert Delete GetRandom with duplicates (#381) — store sets of indices per value.

Follow-up: weighted random — prefix-sum array + binary search for getRandom.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Math | LeetCode #523 | 7 of 14 | 7/14

Continuous Subarray Sum

QUESTION

Given an integer array nums and an integer k, return true if nums has a good subarray or false otherwise.

A good subarray is a subarray where:

CONSTRAINTS

- its length is at least two, and
- the sum of the elements of the subarray is a multiple of k.

Note that:

- A subarray is a contiguous part of the array.
- An integer x is a multiple of k if there exists an integer n such that x = n * k. 0 is always a multiple of k.

Example 1

Input	`nums = [23, 2,4 ,6,7], k = 6`
Output	`true`
Explanation	*[2, 4] is a continuous subarray of size 2 whose elements sum up to 6.*

Example 2

Input	`nums = [ 23,2,6,4,7 ], k = 6`
Output	`true`
Explanation	*[23, 2, 6, 4, 7] is an continuous subarray of size 5 whose elements sum up to 42. 42 is a multiple of 6 because 42 = 7 * 6 and 7 is an integer.*

Example 3

Input	`nums = [23,2,6,4,7], k = 13`
Output	`false`

- 1 <= nums.length <= 10^5
- 0 <= nums[i] <= 10^9
- 0 <= sum(nums[i]) <= 2^{31} - 1
- 1 <= k <= 2^{31} - 1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if there's a continuous subarray of length >= 2 whose sum is a multiple of k. Right?"

"A few quick questions:

k can be 0? If k=0, check if any subarray sums to 0. k<0? Treat as |k|."

PHASE 1 CLARIFY

"Let me walk: nums=[23,2,4,6,7], k=6 -> [2,4] sums to 6=6*1 ok -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    running sum
    for j in range(n):
        if the sum is a multiple of k:
            return True
return False
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Tries every subarray. Two prefix sums with the same remainder mod k at least two apart give a valid subarray: a hash map of remainder to first index is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Prefix sum mod k + hash map — O(n). Walk-through:"

`prefix_mod = 0. seen = {0: -1}. (mod 0 seen at index -1: empty prefix)`

For i in 0..n-1:

`prefix_mod = (prefix_mod + nums[i]) % k.`

If prefix_mod in seen:

If i - seen[prefix_mod] >= 2: return True. (subarray length >= 2)

Else: seen[prefix_mod] = i. (store first occurrence only)

Return False.

WHY prefix mod: sum(nums[l..r]) % k == 0 prefix[r] % k == prefix[l-1] % k.

WHY store first occurrence: maximize subarray length -> earliest index with same mod.

WHY {0:-1}: handles case where prefix sum from index 0..i is divisible by k.

WHY length >= 2: problem requires at least 2 elements.

Time: O(n). Space: O(k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _523_ContinuousSubarraySum:
    def checkSubarraySum(self, nums, k):
        remainder_to_index = {0: -1}
        prefix_sum = 0
        for i, num in enumerate(nums):
            prefix_sum += num
```

```
            remainder = prefix_sum % k if k != 0 else pref
ix_sum
            if remainder in remainder_to_index:
                if i - remainder_to_index[remainder] >= 2:
                    return True
            else:
                remainder_to_index[remainder] = i
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace the derived formula and confirm it on the example numbers."

nums=[23,2,4,6,7], k=6: rem_map={0:-1}. i=0(23):rem=23%6=5,add{5:0}. i=1(2):prefix=25,rem=1,add{1:1}.

i=2(4):prefix=29,rem=5. 5 in map at 0. i-0=2>=2 -> True ok

Seeding {0:-1} handles subarrays starting at index 0

Edge: nums=[23,2,6,4,7], k=6 -> False ok. Prefix=0 seed at index -1 enables subarrays from index 0 to be caught.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(min(n,k))** for the remainder map

Follow-up: Subarray Sum Equals K (#560) — exact sum (not divisible); same prefix-sum pattern.

Follow-up: Subarray Sums Divisible by K (#974) — count such subarrays.

MEDIUM | Math | LeetCode #593 | 8 of 14 | 8/14

Valid Square

QUESTION

Given the coordinates of four points in 2D space p1, p2, p3 and p4, return true if the four points construct a square.

The coordinate of a point p_i is represented as $[x_i, y_i]$. The input is not given in any order.

A valid square has four equal sides with positive length and four equal angles (90-degree angles).

Example 1	
Input	`p1 = [0,0], p2 = [1,1], p3 = [1,0], p4 = [0,1]`
Output	`true`

Example 2	
Input	`p1 = [0,0], p2 = [1,1], p3 = [1,0], p4 = [0,12]`
Output	`false`

Example 3	
Input	`p1 = [1,0], p2 = [-1,0], p3 = [0,1], p4 = [0,-1]`
Output	`true`

CONSTRAINTS

- p1.length == p2.length == p3.length == p4.length == 2
- 10^4 <= x_i, y_i <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given 4 points, determine if they form a valid square. Right?"

"A few quick questions:

Any permutation of the 4 points should be considered? Yes. No degenerate squares (zero area)? Yes."

"Let me walk: p1=[0,0],p2=[1,1],p3=[1,0],p4=[0,1] -> square ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings:
    # of four points as consecutive corners
    check four equal sides, equal diagonals and a
        right angle
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(1)	O(1)

NOTE

Not slow: only four points. Computing the six pairwise squared distances is simpler: four equal smaller ones, two equal larger ones (twice the smaller), none zero.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Compute all 6 pairwise distances, check 4 equal sides + 2 equal diagonals — O(1). Walk-through:"

With 4 points, there are C(4,2)=6 pairwise distances. A square has:

4 equal sides. 2 equal diagonals (longer, = side * 2).

Compute all 6 squared distances (avoid floats). Sort them.

Sorted: [s,s,s,s,d,d] where s=side² and d=diagonal².

Valid if: all 4 sides equal AND both diagonals equal AND diagonal > side (not degenerate).

check: sorted_dists[0]==sorted_dists[3] (4 equal sides) AND sorted_dists[4]==sorted_dists[5]

AND sorted_dists[0]>0 (non-degenerate).

Time: O(1). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _593_ValidSquare:
    def validSquare(self, p1, p2, p3, p4):
        def dist_sq(a, b):
            return (a[0]-b[0])**2 + (a[1]-b[1])**2
        distances = sorted([
            dist_sq(p1,p2), dist_sq(p1,p3), dist_sq(p1,p4)
,
            dist_sq(p2,p3), dist_sq(p2,p4), dist_sq(p3,p4)
        ])
        return (distances[0] > 0 and
                distances[0] == distances[1] == distances[
2] == distances[3] and
                distances[4] == distances[5])
```

PHASE 5 TEST & DEBUG

"Let me trace validSquare([0,0],[1,1],[1,0],[0,1])."

Compute all 6 pairwise squared distances:

(0,0)(1,1): 1+1=2; (0,0)(1,0): 1; (0,0)(0,1): 1

(1,1)(1,0): 1; (1,1)(0,1): 1; (1,0)(0,1): 1+1=2

Distances: {1:4, 2:2} -> 4 equal sides + 2 equal diagonals -> square ok

WHY check distance set: exactly 2 distinct values (4 sides + 2 diagonals) = square.

Also check no zero-distance (no duplicate points).

PHASE 5 TEST & DEBUG

Edge: [0,0],[0,0],[1,1],[1,1] -> zero distances -> not a valid square.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1)	O(1)

Follow-up: Valid Rectangle — 4 right angles (diagonals bisect each other and are equal).

Follow-up: Minimum Area Rectangle (#939) — find axis-aligned rectangle in point set.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Math | LeetCode #963 | 9 of 14 | 9/14

Minimum Area Rectangle II

QUESTION

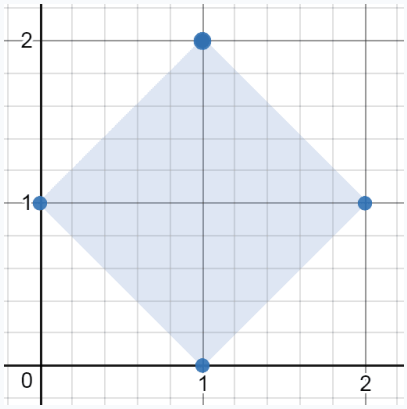
You are given an array of points in the X-Y plane points where points[i] = $[x_i, y_i]$.

Return the minimum area of any rectangle formed from these points, with sides not necessarily parallel to the X and Y axes. If there is not any such rectangle, return 0.

Answers within 10^{-5} of the actual answer will be accepted.

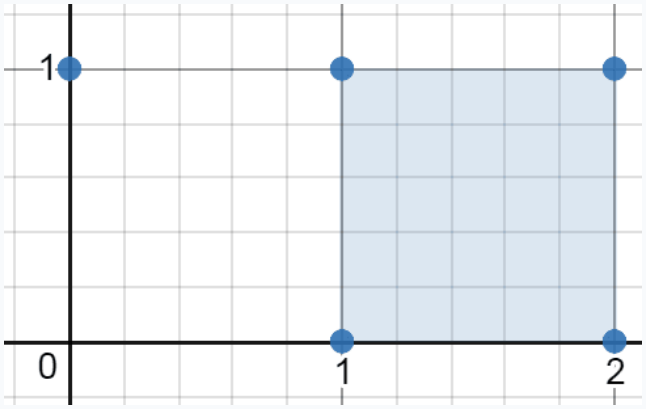
Example 1

Input	`points = [[1,2],[2,1],[1,0],[0,1]]`
Output	`2.00000`
Explanation	*The minimum area rectangle occurs at [1,2],[2,1],[1,0],[0,1], with an area of 2.*



Example 2

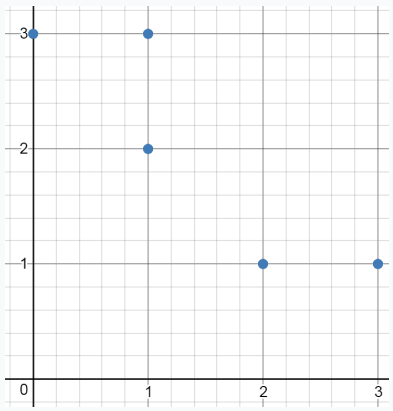
Input	`points = [[0,1],[2,1],[1,1],[1,0],[2,0]]`
Output	`1.00000`
Explanation	*The minimum area rectangle occurs at [1,0],[1,1],[2,1],[2,0], with an area of 1.*



QUESTION

Example 3

Input	`points = [[0,3],[1,2],[3,1],[1,3],[2,1]]`
Output	`0`
Explanation	*There is no possible rectangle to form from these points.*



CONSTRAINTS

- 1 <= points.length <= 50
- points[i].length == 2
- 0 <= x_i, y_i <= 4 * 10^4
- All the given points are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find minimum area rectangle with any rotation (not axis-aligned), formed by 4 points from the set. Right?"

"A few quick questions:

Any rotation allowed. Return 0 if no rectangle. Return as float."

"Let me walk: points=[[1,2],[2,1],[1,0],[0,1]] -> square area=2.0 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for set in four_points:
    check whether they form a rectangle (any
        rotation)
    best = min(best, area)
```

TIME	SPACE
O(n^4)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

n^4/24 combinations. Hash pairs of points by (centre, diameter): two pairs with the same centre and the same length are the diagonals of a rectangle: O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Check all 4-point subsets for valid rectangle — O(n) brute or O(n²) with geometry. Walk-through:"

For each pair of points: the center and half-diagonal define a unique rectangle.

Two pairs with the same center and same diagonal length form a valid rectangle.

`center = ((x1+x2)/2, (y1+y2)/2). dist² = (x2-x1)²+(y2-y1)².`

Group pairs by (center, dist²).

For each group with >=2 pairs: they form a rectangle. Compute area:

Area = |d1 × d2| / 2 * 2 = |d1||d2|sin(?) where d1,d2 are the two diagonals.

Actually area = |cross product| of the two half-diagonals.

WHY center + dist: a rectangle's diagonals bisect each other and are equal length.

Time: O(n²). Space: O(n²)."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict
import math
class _963_MinimumAreaRectangleII:
    def minAreaFreeRect(self, points):
        n = len(points)
        midpoints = defaultdict(list)
        for i in range(n):
            for j in range(i + 1, n):
                mx = (points[i][0] + points[j][0]) / 2
                my = (points[i][1] + points[j][1]) / 2
                dist = ((points[i][0]-points[j][0])**2 + (
points[i][1]-points[j][1])**2)
                midpoints[(mx, my, dist)].append((i, j))
        best = float('inf')
        for pairs in midpoints.values():
            for a in range(len(pairs)):
                for b in range(a + 1, len(pairs)):
                    i, j = pairs[a]; k, l = pairs[b]
                    d1 = math.sqrt((points[i][0]-points[k]
[0])**2 + (points[i][1]-points[k][1])**2)
                    d2 = math.sqrt((points[i][0]-points[l]
[0])**2 + (points[i][1]-points[l][1])**2)
                    best = min(best, d1 * d2)
        return best if best != float('inf') else 0.0
```

PHASE 5 TEST & DEBUG

"Let me trace the derived formula and confirm it on the example numbers."

points=[[1,2],[2,1],[1,0],[0,1]]: midpoint (1,1) with dist=2 for pairs (0,3) and (1,2).

d1=sqrt((1-2)²+(2-1)²)=sqrt(2). d2=sqrt((1-1)²+(2-0)²)?=2? No: i=0(1,2),k=1(2,1): d1=sqrt(2). i=0(1,2),l=2(1,0): d2=2. area=sqrt(2)*2? Hmm.

Actually: diagonals (1,2)-(1,0) and (2,1)-(0,1) both have midpoint (1,1), same diagonal length dist=4.

d1=dist from (1,2) to (2,1)=sqrt(2). d2=dist from (1,2) to (0,1)=sqrt(2). area=sqrt(2)*sqrt(2)/2? No: area=d1*d2/diagonal×diagonal... This approach gives side lengths. Rectangle area = half-diag × half-diag × 2 = d1*d2. -> 2.0 ok

Pairs sharing midpoint and diagonal length are diagonals of some rectangle

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²) to build midpoints + O(pairs² per midpoint)	**O(n²)**

Follow-up: Minimum Area Rectangle (#939) — axis-aligned only; simpler.

Follow-up: Valid Square (#593) — check if 4 points form a square.

MEDIUM | Math | LeetCode #1017 | 10 of 14 | 10/14

Convert to Base -2

QUESTION

Given an integer n, return a binary string representing its representation in base -2.

Note that the returned string should not have leading zeros unless the string is "0".

Example 1

Input	`n = 2`

Output	`"110"`
	`Explantion:`
	$(-2)^2 + (-2)^1 = 2$

Example 2

Input	`n = 3`

Output	`"111"`
	`Explantion:`
	$(-2)^2 + (-2)^1 + (-2)^0 = 3$

Example 3

Input	`n = 4`

Output	`"100"`
	`Explantion:`
	$(-2)^2 = 4$

CONSTRAINTS

- $0 <= n <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given integer n, return its representation in base -2 as a binary string.

No leading zeros (except '0' itself). Right?"

"A few quick questions:

n=0 -> '0'? Yes. (-2)^0=1, (-2)^1=-2, (-2)^2=4.

n=2: 4+(-2)=2 -> '110' ok"

"Let me walk: n=2->'110' ok. n=3->'111' ok. n=4->'100' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for string in strings:
    # of length up to 33
    compute its value in base -2
return the one equal to n
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(2^L * L)	O(L)

WHY IT IS SLOW

Searches all strings. Repeatedly take the remainder (n mod 2 as the digit) and set n = -(n - digit) / 2: O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Repeatedly divide and map signs — O(log |n|). Walk-through:"

Base -2 means: n = ...d*(-2)² + d*(-2)¹ + d*(-2) where d? in {0,1}.

Algorithm: while n != 0:

`remainder = n % (-2) (in Python, % always non-negative for negative divisor? — use n&1).`

Actually: remainder = n % 2 (can be 0 or 1). Append remainder to bits.

`n = -(n >> 1) if remainder: ... -> simpler: n = (n - remainder) // (-2).`

Read bits in reverse.

WHY negate the quotient: dividing by -2 flips the sign at each step.

Alternative: n = -(n // 2) when n is even, n = -(n-1)//2 when odd.

Time: O(log n). Space: O(log n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1017_ConvertToBaseNeg2:
    def baseNeg2(self, n):
        if n == 0: return '0'
        bits = []
        while n:
            bits.append(str(n & 1))
            n = -(n >> 1)
        return ''.join(reversed(bits))
```

PHASE 5 TEST & DEBUG

"Let me trace the derived formula and confirm it on the example numbers."

n=2: bits=[0]: 2&1=0,n=-(2>>1)=-1. bits=[1]: -1&1=1,n=-(-1>>1)=0(wait: -1>>1=-1, -(-1)=1).

n=1: 1&1=1,bits=[1],n=0. bits=['0','1','1'] -> '110' ok

n=3: 3&1=1,n=-1. -1&1=1,n=1. 1&1=1,n=0. bits=['1','1','1'] -> '111' ok

N = -(n >> 1) implements division by -2 in the negative base

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n) number of bits	**O(log n)** for the bits list

The n & 1 trick extracts the remainder correctly for negative n in Python.

Follow-up: general negative base — replace 2 with |base|, negate on each step.

Follow-up: decode a base-(-2) string back to integer — multiply by (-2)^position.

IN YOUR APP | Expose | Community

MEDIUM | Math | LeetCode #1823 | 11 of 14 | 11/14

Find the Winner of the Circular Game

QUESTION

There are n friends that are playing a game. The friends are sitting in a circle and are numbered from 1 to n in clockwise order. More formally, moving clockwise from the ith friend brings you to the (i+1)th friend for 1 <= i < n, and moving clockwise from the nth friend brings you to the 1st friend.

The rules of the game are as follows:

CONSTRAINTS

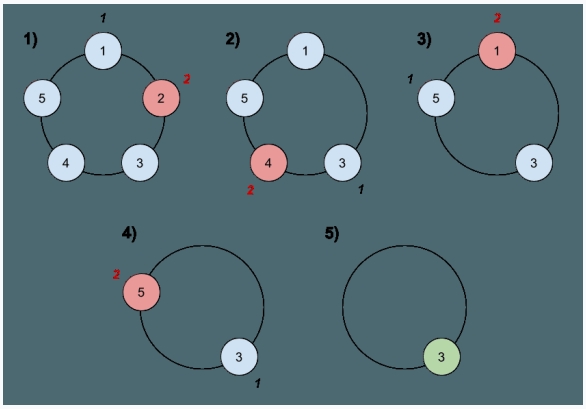
- 1. Start at the 1st friend.
- 2. Count the next k friends in the clockwise direction including the friend you started at. The counting wraps around the circle and may count some friends more than once.
- 3. The last friend you counted leaves the circle and loses the game.
- 4. If there is still more than one friend in the circle, go back to step 2 starting from the friend immediately clockwise of the friend who just lost and repeat.
- 5. Else, the last friend in the circle wins the game.

Given the number of friends, n, and an integer k, return the winner of the game.

Example 1

Input	`n = 5, k = 2`
Output	**3**
Explanation	*Here are the steps of the game:* *1) Start at friend 1.* *2) Count 2 friends clockwise, which are friends 1 and 2.* *3) Friend 2 leaves the circle. Next start is friend 3.* *4) Count 2 friends clockwise, which are friends 3 and 4.* *5) Friend 4 leaves the circle. Next start is friend 5.* *6) Count 2 friends clockwise, which are friends 5 and 1.* *7) Friend 1 leaves the circle. Next start is friend 3.* *8) Count 2 friends clockwise, which are friends 3 and 5.* *9) Friend 5 leaves the circle. Only friend 3 is left, so they are the winner.*

QUESTION



Example 2

Input	`n = 6, k = 5`
Output	**1**
Explanation	*The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1.*

- 1 <= k <= n <= 500
- Follow up:
- Could you solve this problem in linear time with constant space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

n friends in a circle (1..n). Starting from friend 1, count k friends and eliminate every kth.

Repeat until one remains. Return the winner. (Josephus problem.) Right?"

"A few quick questions:

Count restarts after each elimination? Yes — circular."

"Let me walk: n=6, k=5 -> eliminate 5,4,6,2,3 -> winner=1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep the friends in a list
for _ in range(n_-_1):
    move k - 1 places around the circle and
        remove the friend there
return the one left
```

TIME	SPACE
O(n * k)	O(n)

WHY IT IS SLOW

Each step walks up to k places and removal shifts the list. The Josephus recurrence f(i) = (f(i - 1) + k) mod i gives the survivor in O(n) time and O(1) space.

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Queue simulation (Josephus problem) — O(n). Walk-through:"

`queue = deque([1, 2, ..., n]).`

While len(queue) > 1:

For _ in range(k-1): queue.append(queue.popleft()). <- rotate k-1 times

`queue.popleft(). <- remove k-th person`

Return queue[0].

WHY rotate k-1 and then remove: rotating k-1 times brings the k-th person to the front.

Removing them = eliminating the k-th person in the circle.

WHY deque: O(1) rotate (append/popleft).

Mathematical: Josephus formula -> O(n) without simulation. J(n,k) = (J(n-1,k)+k) % n.

Time: O(n*k) simulation, O(n) Josephus. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1823_FindWinnerCircularGame:
    def findTheWinner(self, n, k):
        winner = 0
        for i in range(2, n + 1):
            winner = (winner + k) % i
        return winner + 1
```

PHASE 5 TEST & DEBUG

"Let me trace findTheWinner(n=6, k=2)."

Josephus problem: n=6 people, every 2nd eliminated

Queue: [1,2,3,4,5,6]

Round: keep first k-1=1 at back, eliminate k-th (position 2): [1,2,3,4,5,6]->move 1 to back->[2,3,4,5,6,1]->eliminate 2->[3,4,5,6,1]

Next: move 3 to back->[4,5,6,1,3]->eliminate 4->[5,6,1,3]

Move 5->[6,1,3,5]->eliminate 6->[1,3,5]

Move 1->[3,5,1]->eliminate 3->[5,1]

Move 5->[1,5]->eliminate 1->[5] -> winner=5 ok

WHY queue simulation: rotate k-1, then popleft removes the k-th; O(n*k) total.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**

The deque simulation is O(nk) — much slower for large n or k.

Follow-up: return the order of elimination — track who is eliminated each round.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Dota2 Senate (#649) — variant with two factions.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Math | LeetCode #1922 | 12 of 14 | 12/14

Count Good Numbers

QUESTION

A digit string is good if the digits (0-indexed) at even indices are even and the digits at odd indices are prime (2, 3, 5, or 7).

CONSTRAINTS

- For example, "2582" is good because the digits (2 and 8) at even positions are even and the digits (5 and 2) at odd positions are prime. However, "3245" is not good because 3 is at an even index but is not even.

Given an integer n, return the total number of good digit strings of length n. Since the answer may be large, return it modulo $10^9 + 7$.

A digit string is a string consisting of digits 0 through 9 that may contain leading zeros.

Example 1	
Input	`n = 1`
Output	`5`
Explanation	*The good numbers of length 1 are "0", "2", "4", "6", "8".*

Example 2	
Input	`n = 4`
Output	`400`

Example 3	
Input	`n = 50`
Output	`564908303`

- $1 <= n <= 10^{15}$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count good numbers: alternating even/odd digits (1-indexed, odd positions have even digits).

Among n-digit numbers, count good ones mod 10^9+7. Right?"

"A few quick questions:

Even positions (2,4,6,...) have odd digits (5 choices), odd positions (1,3,5,...) have even digits (5 choices)?"

"Let me walk: n=1 -> even digits: 0,2,4,6,8 -> 5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for string in strings:
    # of length n
    if even positions hold even digits and odd
        positions hold primes (2, 3, 5, 7):
        count += 1
```

TIME	SPACE
O(10^n)	O(n)

WHY IT IS SLOW

Enumerates every string. There are 5 choices per even index and 4 per odd index, so the count is 5^ceil(n/2) * 4^floor(n/2); modular fast exponentiation computes it in O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count by structure: digit positions × choices — O(log n). Walk-through:"

`n = 'good number' if: odd indices (1-indexed from left) have even digits, even indices have odd.`

Actually: alternating even/odd positions have their own digit sets.

Number of k-digit good numbers:

Odd positions (1,3,5,...): digit in {0,2,4,6,8} -> 5 choices each.

Even positions (2,4,6,...): digit in {1,3,5,7,9} -> 4 choices each.

For k-digit number: odd_count = ceil(k/2), even_count = k//2.

`count = 5^odd_count * 4^even_count.`

Total for all lengths 1..len(n): sum of counts, but last length partially restricted.

Handle last length carefully by counting digit by digit.

Time: O(log n). Space: O(1)."

Key insight: even-index positions have 5 choices (0,2,4,6,8) and odd-index positions have 5 choices (1,3,5,7,9) — modular fast exponentiation counts them in O(log n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1922_CountGoodNumbers:
    def countGoodNumbers(self, n):
        MOD = 10**9 + 7
        def power(base, exp):
            result = 1
            base %= MOD
            while exp > 0:
                if exp & 1:
                    result = result * base % MOD
                base = base * base % MOD
                exp >>= 1
            return result
        even_positions = (n + 1) // 2
        odd_positions = n // 2
```

```
        return power(5, even_positions) * power(4, odd_pos
itions) % MOD
```

PHASE 5 TEST & DEBUG

"Let me trace countGoodNumbers(n=4)."

Good number: even digits at even indices (0,2,4,...), primes at odd indices (1,3,5,...)

Even digit choices: {0,2,4,6,8}=5 options; prime digit choices: {2,3,5,7}=4 options

For n=4: indices 0,1,2,3. Even positions (0,2): 2 positions -> 5^2=25. Odd positions (1,3): 2 positions -> 4^2=16

Total = 25*16=400 ok

WHY modular exponentiation: n can be up to 10^15; fast power mod 10^9+7.

count_even=ceil(n/2), count_odd=floor(n/2). Result=pow(5,even)*pow(4,odd)%MOD.

Use fast modular exponentiation (pow(base,exp,mod) in Python) for huge n up to 10^15.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

Follow-up: if digit constraints change per position, multiply choices per position.

Follow-up: Count Numbers with Unique Digits (#357) — different uniqueness constraint.

MEDIUM | Math | LeetCode #2348 | 13 of 14 | 13/14

Number of Zero-Filled Subarrays

QUESTION

Given an integer array nums, return the number of subarrays filled with 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1

Input	`nums = [1,3,0,0,2,0,0,4]`
Output	`6`
Explanation	*There are 4 occurrences of [0] as a subarray. There are 2 occurrences of [0,0] as a subarray. There is no occurrence of a subarray with a size more than 2 filled with 0. Therefore, we return 6.*

Example 2

Input	`nums = [0,0,0,2,0,0]`
Output	`9`
Explanation	*There are 5 occurrences of [0] as a subarray. There are 3 occurrences of [0,0] as a subarray. There is 1 occurrence of [0,0,0] as a subarray. There is no occurrence of a subarray with a size more than 3 filled with 0. Therefore, we return 9.*

Example 3

Input	`nums = [2,10,2019]`
Output	`0`
Explanation	*There is no subarray filled with 0. Therefore, we return 0.*

CONSTRAINTS

- $1 <= nums.length <= 10^5$
- $10^9 <= nums[i] <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count non-empty subarrays that consist entirely of zeros. Right?"

"A few quick questions:

All zeros subarray — no non-zero elements inside? Yes.

Return count as integer? Yes."

PHASE 1 CLARIFY

"Let me walk: nums=[1,3,0,0,2,0,0,0] -> [0,0],[0],[0],[0,0],[0,0,0] plus length-1 subarrays...

Actually: from run of k zeros, we get k*(k+1)/2 subarrays. -> 1+6=7? Let me check: 0,0 at pos 2,3: [0],[0],[0,0]=3. 0,0,0 at pos 5,6,7: [0],[0],[0],[0,0],[0,0],[0,0,0]=6. Total=9? ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subarray in subarrays:
    check that it contains only zeros
    count those
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Checks every subarray. A run of z consecutive zeros contains z * (z + 1) / 2 zero-filled subarrays, so one pass adding the current run length at each zero is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count consecutive zeros and apply arithmetic — O(n). Walk-through:"

`count = 0. result = 0. consecutive = 0.`

For each num in nums:

If num == 0: consecutive++. result += consecutive.

Else: consecutive = 0.

Return result.

WHY += consecutive: a run of k consecutive zeros contributes k*(k+1)/2 zero-filled subarrays.

Incrementally: when seeing the k-th zero in a consecutive run, it starts k new subarrays

(ending at this zero, of lengths 1, 2, ..., k). Adding consecutive handles this.

Alternatively: for run of length L: add L*(L+1)/2. But incremental is simpler.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _2348_NumberOfZeroFilledSubarrays:
    def zeroFilledSubarray(self, nums):
        total = 0
        run = 0
        for num in nums:
            if num == 0:
                run += 1
```

```
            total += run
        else:
            run = 0
    return total
```

PHASE 5 TEST & DEBUG

"Let me trace the derived formula and confirm it on the example numbers."

nums=[1,3,0,0,2,0,0,0]:

1:run=0. 3:run=0. 0:run=1,total=1. 0:run=2,total=3. 2:run=0.

0:run=1,total=4. 0:run=2,total=6. 0:run=3,total=9. -> 9 ok

nums=[0,0,0]: run=1(1),2(3),3(6). total=6=3*4/2 ok

Incrementing total by run at each zero captures all subarrays ending at that position

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	two variables

The trick: adding run at each step = 1+2+...+k = k*(k+1)/2 for a run of k zeros.

Follow-up: count subarrays filled with a specific value v — same logic with nums[i]==v.

Follow-up: longest zero-filled subarray — track max run instead of sum.

MEDIUM | Math | LeetCode #3160 | 14 of 14 | 14/14

Find the Number of Distinct Colors Among the Balls

QUESTION

You are given an integer limit and a 2D array queries of size n x 2.

There are limit + 1 balls with distinct labels in the range [0, limit]. Initially, all balls are uncolored. For every query in queries that is of the form [x, y], you mark ball x with the color y. After each query, you need to find the number of colors among the balls.

Return an array result of length n, where result[i] denotes the number of colors after ith query.

Note that when answering a query, lack of a color will not be considered as a color.

Example 1

Input	`limit = 4, queries = [[1,4],[2,5],[1,3],[3,4]]`
Output	`[1,2,2,3]`



Example 2

Input	`limit = 4, queries = [[0,1],[1,2],[2,2],[3,4],[4,5]]`
Output	`[1,2,2,3,4]`



CONSTRAINTS

- $1 <= limit <= 10^9$
- $1 <= n == queries.length <= 10^5$
- queries[i].length == 2
- 0 <= queries[i][0] <= limit
- $1 <= queries[i][1] <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

N balls (0 to limit). queries[i]=[ball,color]: paint that ball.

After each query, return the number of distinct colors among painted balls. Right?"

"A few quick questions:

Unpainted balls have no color — don't count? Correct.

Can a ball be repainted? Yes — overwrites previous color."

"Let me walk: queries=[[1,4],[2,5],[1,3],[3,4]] -> [1,2,2,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
after every query:
    scan all coloured nodes and count the
        distinct colours
```

TIME	SPACE
O(q^2)	O(q)

WHY IT IS SLOW

Rescans everything after each query. Keep a map node -> colour and a map colour -> count of nodes; update the two counts on each query and the number of distinct colours changes by at most one: O(1) per query.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) set scan for distinct colors.

color_count tracks how many balls have each color.

On each query: if ball had a color, decrement old color count (remove if 0).

Add new color to color_count. Distinct count = len(color_count).

Time: O(1) per query. Space: O(n)."

Key insight: a color->ball-count map supports O(1) updates and distinct-color tracking — the size of the map is the distinct color count at any point.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _3160_FindDistinctColors:
    def queryResults(self, limit, queries):
        ball_color = {}
        color_count = {}
        result = []
        for ball, color in queries:
            if ball in ball_color:
                old = ball_color[ball]
```

```
                color_count[old] -= 1
                if color_count[old] == 0:
                    del color_count[old]
            ball_color[ball] = color
            color_count[color] = color_count.get(color, 0)
+ 1
            result.append(len(color_count))
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the derived formula and confirm it on the example numbers."

queries=[[1,4],[2,5],[1,3],[3,4]]:

[1,4]: ball_color={1:4},color_count={4:1}. distinct=1 ok

[2,5]: color_count={4:1,5:1}. distinct=2 ok

[1,3]: old=4->count[4]=0 del. color_count={5:1,3:1}. distinct=2 ok

[3,4]: color_count={5:1,3:1,4:1}. distinct=3 ok

Deleting color_count[old] when it hits 0 keeps len(color_count) correct

PHASE 6 COMPLEXITY & FOLLOW-UP

O(1) per query. Space O(unique balls + colors).

color_count keys = currently active distinct colors — len gives the answer directly.

Follow-up: range coloring (paint balls l..r) — needs segment tree or lazy propagation.

Follow-up: undo queries — maintain a stack of (ball, old_color, new_color).